

ITSOLERA PVT LTD

Summer Internship — Offensive Security (Tool Development)

Recon Forge

Modular Reconnaissance Tool

Author:

Muhammad Faizan

B.S. Cybersecurity — MUET Jamshoro

Task 1 — Custom Reconnaissance Tool Development

Deadline: 10th July 2026

1. Overview

ReconForge is a lightweight, modular CLI reconnaissance tool built in Python for the passive and active information-gathering phases of an authorized penetration test. Every capability is implemented as an independent module and wired into the CLI through command-line flags, so individual modules can be run in isolation or combined for a full recon run.

⚠ Authorized use only. This tool performs active port scanning and banner grabbing. Only run it against domains/hosts you own or have explicit written permission to test.

2. Features

Run everything in one shot with `--all` (also the default if no module flag is given).

Category	Capability	Flag
Passive Recon	WHOIS lookup	<code>--whois</code>
Passive Recon	DNS enumeration (A, MX, TXT, NS)	<code>--dns</code>
Passive Recon	Subdomain enumeration (crt.sh + OTX + HackerTarget + RapidDNS + DNS Brute-force)	<code>--subdomains</code>
Active Recon	TCP port scanning (socket-based, threaded)	<code>--ports</code>
Active Recon	Banner grabbing on open ports	<code>--banner</code>
Active Recon	Web technology detection	<code>--techdetect</code>
Reporting	Generates .txt and .html reports with timestamps and IP resolution	<code>--format</code>
Modularity	Each module runs independently with leveled logging	<code>-v / -vv</code>

3. Project Architecture

Each module exposes a `run(...)` function returning a plain Python dict, so modules are independently testable and importable. The CLI layer stays thin — pure orchestration and argument parsing.

File / Folder	Purpose
<code>recontool.py</code>	CLI entrypoint — argument parsing, orchestration, logging
<code>modules/whois_lookup.py</code>	WHOIS lookup via <code>python-whois</code> with raw socket fallback
<code>modules/dns_enum.py</code>	DNS records (A, MX, TXT, NS) via <code>dnspython</code>
<code>modules/subdomain_enum.py</code>	4 APIs + DNS brute-force fallback (80+ wordlist)
<code>modules/port_scan.py</code>	Threaded TCP <code>connect()</code> scan + banner grabbing
<code>modules/tech_detect.py</code>	HTTP header and HTML signature fingerprinting
<code>modules/report.py</code>	.txt and .html report generation with timestamps
<code>requirements.txt</code>	Python dependencies (<code>dnspython</code> , <code>python-whois</code>)
<code>Dockerfile</code>	Containerized deployment (bonus deliverable)
<code>reports/</code>	Generated reports saved here by default

. Installation

Step 1 — Extract the ZIP

Download recontool.zip and extract it. On Windows: right-click → Extract All. On Linux:

```
unzip recontool.zip
cd recontool
```

Step 2 — Verify Python Version

Python 3.8 or higher is required. Check your version:

```
python --version
```

Step 3 — Install Dependencies

Navigate into the recontool folder and run:

```
pip install -r requirements.txt
```

This installs exactly 2 external libraries:

- dnspython — DNS record enumeration (A, MX, TXT, NS records)
- python-whois — Structured WHOIS data parsing

All other modules use Python built-in libraries only (socket, urllib, ssl, concurrent.futures, argparse, logging).

Step 4 — Verify Installation

```
python -c "import dns; import whois; print('Both libraries installed successfully!')"
```

5. Usage

Running from Command Prompt / Terminal

```
python recontool.py eccouncil.org --all
python recontool.py eccouncil.org --whois --dns
python recontool.py eccouncil.org --subdomains -v
python recontool.py eccouncil.org --ports --ports-range 1-1000 --banner
python recontool.py eccouncil.org --all --format html --outdir my_reports
python recontool.py eccouncil.org --all -vv
```

Running from PyCharm (Click Run)

Add the following line inside the if `__name__ == '__main__':` block before `main()`:

```
sys.argv = ["recontool.py", "eccouncil.org", "--all", "-v"]
```

Change eccouncil.org to your target domain. Remove this line when running from CMD.

6. CLI Reference

Flag	Group	Description
--whois	Passive Recon	Perform WHOIS lookup
--dns	Passive Recon	Enumerate DNS records (A, MX, TXT, NS)
--subdomains	Passive Recon	Enumerate subdomains via 4 APIs + DNS brute-force
--ports	Active Recon	Perform TCP port scan
--ports-range RANGE	Active Recon	Port range, e.g. 1-1024 (default: 1-1024)
--banner	Active Recon	Grab service banners on open ports
--techdetect	Active Recon	Detect web technologies (HTTP headers)
--all	General	Run all recon modules
--format {txt,html,both}	General	Report output format (default: both)
--outdir OUTDIR	General	Directory to write report (default: reports/)
-v / -vv	General	Verbosity: INFO / DEBUG level logging
--version	General	Show tool version

. Logging & Verbosity

ReconForge implements leveled logging using Python's built-in logging module. The verbosity level controls how much output you see in the terminal during a scan.

Flag	Level	What You See
(no flag)	WARNING	Only errors and critical warnings — silent run
-v	INFO	Module progress, record counts, file paths saved
-vv	DEBUG	Every request detail, fallback paths, per-port results

How Verbosity Works in Code

The `configure_logging()` function force-sets the level on the root logger and all its handlers directly, ensuring it works correctly in both CMD and PyCharm environments.

8. Module Details

8.1 WHOIS Lookup (whois_lookup.py)

- Tries the python-whois library first for structured, parsed output

- Returns: registrar, creation date, expiry date, name servers, org, country, emails
- Fallback: raw TCP socket query on port 43 against IANA WHOIS referral chain
- Degrades gracefully — never crashes even if both methods fail

8.2 DNS Enumeration (`dns_enum.py`)

- Enumerates A, MX, TXT, and NS records using dnspython
- Falls back to `socket.gethostbyname()` for basic A record if dnspython is missing
- Handles NXDOMAIN, NoAnswer, and timeout errors cleanly

8.3 Subdomain Enumeration (`subdomain_enum.py`)

- Queries 4 independent public sources in parallel:
 1. crt.sh — Certificate Transparency log search (with 3 retries on 502/503)
 2. AlienVault OTX — Passive DNS database
 3. HackerTarget — Free DNS lookup API
 4. RapidDNS — Fast subdomain search
- Automatic DNS brute-force fallback: if all 4 APIs fail, tries 80+ common subdomain names via direct DNS resolution (mail, admin, ftp, vpn, exam, library, etc.)
- Base domain is used automatically — www. is stripped from target before querying
- Python 3.9 compatible — no str | None type hint syntax

8.4 Port Scanning (`port_scan.py`)

- Multi-threaded TCP connect() scan using Python socket + concurrent.futures
- No nmap dependency — works on any machine with Python
- Configurable port range via `--ports-range` flag (default: 1-1024)
- Up to 100 concurrent threads for fast scanning
- Identifies common services: HTTP, HTTPS, SSH, FTP, SMTP, MySQL, RDP, etc.

8.5 Banner Grabbing (`port_scan.py` — `grab_banners()`)

- Opens a TCP connection to each discovered open port
- Sends an HTTP HEAD request on HTTP ports (80, 8080, 8000) to trigger a response
- Captures and records the first 1024 bytes of the server's response
- Handles timeout gracefully — reports 'no banner' rather than crashing

8.6 Technology Detection (`tech_detect.py`)

- Fetches the target URL over HTTPS first, falls back to HTTP
- Inspects HTTP response headers: Server, X-Powered-By, Set-Cookie
- Scans HTML body for known CMS/framework signatures
- Detects: Nginx, Apache, Cloudflare, IIS, PHP, ASP.NET, WordPress, Joomla, Drupal, Shopify, Next.js, Angular, React, Bootstrap
- No API key required — fully signature-based

8.7 Report Generation (report.py)

- Generates both .txt (plain text) and .html (styled dark-theme) reports
- Every report includes: target domain, resolved IP, UTC timestamp, per-module sections
- Port scan section includes scan start/finish timestamps
- HTML report uses a dark cybersecurity-themed design with teal accent colours
- Reports are auto-named: recon_<domain>_<timestamp>.txt/.html

10. Docker Deployment (Bonus)

ReconForge includes a Dockerfile for containerized deployment. Docker packages the tool, Python runtime, and all dependencies into a single container image, eliminating setup requirements on the host machine.

Build the Image

```
docker build -t reconforge .
```

Run a Scan

```
docker run --rm -v $(pwd)/reports:/app/reports reconforge eccouncil.org --all
```

Reports are saved to your local reports/ folder via the Docker volume mount (-v flag).

11. Design Choices

- Port scanning uses socket + ThreadPoolExecutor instead of shelling out to nmap, making the tool portable and dependency-free for active recon.
- Technology detection uses signature matching against HTTP headers and HTML body instead of the Wappalyzer API, removing the need for any API key.

- WHOIS module has a two-layer fallback: python-whois library first, then raw TCP socket on port 43 against the IANA WHOIS referral chain.
- Subdomain enumeration uses 4 independent public sources plus a DNS brute-force wordlist fallback, ensuring results even on restricted networks.
- Logging uses force-level assignment on the root logger (not basicConfig) to work correctly in both terminal and IDE environments like PyCharm.

12. Disclaimer

This tool is built strictly for educational and authorized penetration-testing purposes as part of the ITsolera Offensive Security internship task. Unauthorized scanning of systems you do not own or have explicit permission to test may violate computer-misuse laws in your jurisdiction. Always obtain written authorization before scanning any target.